

Monitoring Application Progress

Sure! Let's talk about the Spark Application Workflow, which is a key concept in monitoring application progress.

In simple terms, the Spark Application Workflow is like a series of tasks that a chef follows to prepare a meal. Just as a chef has to complete certain steps in a specific order—like chopping vegetables, boiling water, and then cooking the ingredients—Spark breaks down a job into smaller parts called stages. Each stage has its own tasks, and they must be completed before moving on to the next stage. If something goes wrong, like a pot boiling over, the chef needs to quickly identify the problem and fix it to keep cooking. Similarly, Spark helps you monitor these tasks to quickly find and resolve any issues that may arise during the application process.

For example, imagine you are baking a cake. You first mix the ingredients (Stage 0), then pour the batter into a pan (Stage 1), and finally bake it in the oven (Stage 2). If the oven stops working, you need to figure out why and fix it before you can enjoy your cake. In Spark, if a task fails, it will try to fix it a few times, but if it can't, it will mark the job as failed, just like a chef would stop if the cake can't be baked.

Why Monitor an Application

Monitoring an application using the Spark Application UI provides these benefits:

- Quickly identify failed jobs and tasks
- Fast access to locate inefficient operations
- Application workflow optimization

- Running a Spark application can sometimes take a long time and have many possible points of failure. When an issue arises, say a faulty worker node is causing the occasional task to fail, it is essential to find and address the problem quickly, so that cluster resources do not stay idle. The Spark Application UI is a great way to monitor a running application. The application UI centralizes critical information, including status information, and organizes information logically, resulting in convenient and fast access. You can quickly identify failures, then drill down to the lowest levels of the application to find out the root causes of failures. Besides failures, the UI can also help you quickly locate and analyze application processing bottlenecks.

How Does the Application Flow

Multiple related jobs

- From different data sources
- One or more DataFrames
- Actions applied to the DataFrames

Workflows can include:

- Jobs created by the SparkContext in the driver program
- Jobs in progress running as tasks in the executors
- Completed jobs transferring results back to the driver or writing to disk

- A Spark application can consist of many parallel, and often, related jobs, including multiple jobs resulting from multiple data sources, multiple DataFrames, and the actions applied to the DataFrames. Workflows can include: Jobs created by the SparkContext in the driver program, jobs in progress running as tasks in the executors, and completed jobs transferring results back to the driver or writing to disk.

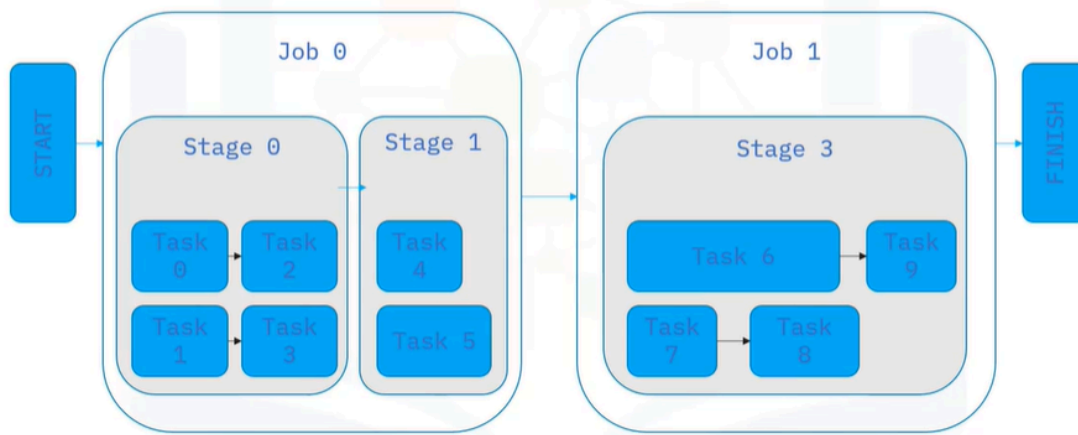
How Do Jobs Progress?

1. Spark jobs divide into stages, which connect as a directed acyclic graph (DAG)
2. Tasks for the current stage are scheduled on the cluster
3. As the stage completes all its tasks, the next dependent stage in the DAG begins
4. The job continues through the DAG until all stages complete

If any tasks within a stage fail, after several attempts, Spark marks the task, stage, and job as failed and stops the application

- First, Spark jobs divide into stages, which connect as a Directed Acyclic Graph, or DAG. Tasks for the current stage are scheduled on the cluster. When the stage completes all of its tasks, the next dependent stage in the DAG begins. The job progresses through the DAG all stages are completed. If any of the tasks within a stage fail, after several attempts, Spark marks the task, stage, and job as failed and stops the application.

Workflow Sequence



- Let's look at an example. The application first creates a job. Next, Spark divides the job into one or more stages. The first stage, Stage "0," has no dependencies, so Spark sends its tasks to the executors. Stage "0" now has two tasks started.
- The width of the task indicates the elapsed run time. Two more tasks that are dependent on "0" and "1" run, but do not require a shuffle, such as map operations, for example, so they remain part of Stage "0" and run independently. The end of Stage "0" now marks the beginning of the next Stage "1."
- This boundary exists because a shuffle was required, which means that all tasks for Stage "1" must wait for all tasks in Stage "0" to finish before starting. Here you can see that tasks 4 and 5 have different run times. Their run times differ because each task is running independently and on different data partitions. However, the stage is not complete until all tasks have finished. With this job completed, Spark can begin a new job.
- In this example, Job 1 depends on the data from Job 0. The next job consists of one stage and starts Tasks 6 and 7. Since tasks within a stage can run independently, when Task 7 completes, the executor running Task 7 can immediately start Task 8 while Task 6 continues to run. When the application completes Tasks 8 and 9, the stage and job are complete, marking the end of this application workflow.

Application Code Example

Monitoring a PySpark application

```
# Read a Parquet file from disk
df = spark.read.parquet("users.parquet")

# Select certain columns and cache to memory
df = df.select("country", "salary").cache()

# Group data by country and compute mean salary per country
mean_salaries = df.groupBy("country").agg({"salary": "mean"}).collect()
```

- Next, view this example application to see how the code translates to a workflow you can monitor using the UI. This application's single data source is a Parquet file loaded from disk to produce a DataFrame. Using that same DataFrame, two columns are selected. The caching action is specific to this example. The application groups the data by the "country" column and then aggregates the data by calculating the mean of the "salary" column. Next the "collect" action runs. This action triggers the job creation and schedules the tasks, as the previous operations are all lazily computed.

Application Jobs Example

The Spark Application UI Jobs tab displays two jobs

- Job 1: Reads the Parquet file
- Job 2: Performs the action to collect the computations to send to the driver

Job Id	Description	Submitted	Duration	Stages: Succeeded/Total	Tasks (for all stages): Succeeded/Total
1	collect at log4spark-3.1.1-bin-hadoop2.7/hog_example.py:213	2021/03/23 23:37:12	7 s	2/2	203/203
0	parquet at NativeMethodAccessorImpl.java:0	2021/03/23 23:37:05	3 s	1/1	3/3

- After you submit the application, start the Spark Application UI and view the Jobs tab, which displays two jobs. One job reads the Parquet file from disk. The second job is the result of the action to collect the grouped aggregate computations to send to the driver.

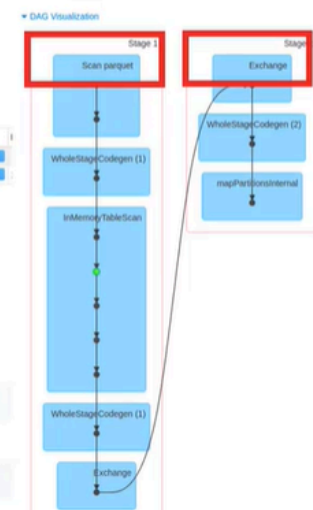
Application Job Details Example

- View the Job Details tab for second job

Stage Id	Description	Submitted	Duration	Tasks: Succeeded/Total
2	collect at log4spark-3.1.1-bin-hadoop2.7/hog_example.py:213	2021/03/23 23:37:14	5 s	200/200
1	collect at log4spark-3.1.1-bin-hadoop2.7/hog_example.py:213	2021/03/23 23:37:12	2 s	3/3

Page: 1

- Job 1 has two stages
- The DAG shows that there is a shuffle between stages



- On the Jobs tab, click a specific job to display its Job Details page. Here you can see the number of stages and the DAG that links the stages. This example has two stages connected by a shuffle exchange, which is due to grouping the data by country in the application.

Application Stage Details Example

Select a stage to view the tasks

See which tasks are scheduled, their duration, and other task states



Application Stage Details Example

Select a stage to view the tasks

Stage Id	Description	Submitted	Duration	Tasks: Succeeded/Total	Input	Output	Shuffle Read	Shuffle Write
2	collect at ipyspark-3.1.1-bin-hadoop2.7/wg_example.py:213	2021/03/23 23:37:14	5 s	200/200			8.8 KB	
1	collect at ipyspark-3.1.1-bin-hadoop2.7/wg_example.py:213	2021/03/23 23:37:12	2 s	5/5	110.7 KB			8.8 KB

Page: 1, 1 Pages, Jump to: 1, Show: 100 items in a page, Go

- Select a stage to view the tasks. The Stage Details timeline indicates each task's state using color coding.
- View the timeline to see when each task was started and the task's duration. Use this information to quickly locate failed tasks, see which tasks are taking a long time to run, and determine how well your application is parallelized.

Application Task Status Example

- View the task status, duration, and amount of data transferred
- Access the executor logs for that task

Task ID	Attempt	Status	Locality level	Executor ID	Host	Logs	Launch Time	Duration	GC Time	Shuffle Read Size / Records	Errors
97	0	SUCCESS	PROCESS_LOCAL	0	192.168.122.182	stdout stderr	2021-03-23 23:46:06	10.0 ms			
2	0	SUCCESS	NODE_LOCAL	0	192.168.122.182	stdout stderr	2021-03-23 23:46:04	0.3 s		80 B / 1	
3	0	SUCCESS	NODE_LOCAL	0	192.168.122.182	stdout stderr	2021-03-23 23:46:04	0.3 s		110 B / 2	
98	0	SUCCESS	PROCESS_LOCAL	0	192.168.122.182	stdout stderr	2021-03-23 23:46:06	40.0 ms			
99	0	SUCCESS	PROCESS_LOCAL	0	192.168.122.182	stdout stderr	2021-03-23 23:46:06	46.0 ms			
100	0	SUCCESS	PROCESS_LOCAL	0	192.168.122.182	stdout stderr	2021-03-23 23:46:06	13.0 ms			

- The task list provides even more metrics, including status, duration, and amount of data transferred as part of a shuffle. Here, you see two tasks that read one and two records as part of a shuffle. You see these two tasks because, by default for a shuffle, Spark repartitions the data into a larger number of partitions. You can also access a task's executor logs. The data used in this example is small. Therefore, many tasks only have a small number of records to process.

Application Completion Example

- The application jobs complete
 - The SparkContext stops
 - The application UI is no longer available
 - If event logging is enabled, view the application UI by starting the Spark History Server and connecting to it
-
- When all application jobs are complete and the results are sent to the driver or written to disk, the SparkContext can be stopped either manually or automatically when the application exits. When the application UI server shuts down the UI is no longer available. To view the Application UI after the application stops, event logging must be enabled. This means that all events in the application workflow are logged to a file, and the UI can be viewed with the Spark History server.

Viewing UI with History Server

Before the application is started, verify that event logging is enabled

```
spark.eventLog.enabled true  
spark.eventLog.dir <path-for-log-files>
```

View the application UI by connecting to Spark History Server

```
# Command to connect to the Spark application UI history server  
http://<host-url>:18080
```

Start the History server

```
# Command to start the History Server  
./sbin/start-history-server.sh
```

- To view the Application UI with the History Server, first verify that event logging is enabled. Enter the event log path as seen using the properties displayed onscreen before submitting the application. When the application completes, the Application UI populates the log files in the event log directory. To start the history server, apply the command shown onscreen. Once the history server is started, connect to the history server by typing the history server host URL followed by the default port number 18080. You can see a list of completed applications and select one to view its application UI.

Summary

In this video, you learned that:

- The Spark application workflow includes jobs created by the SparkContext in the driver program, jobs in progress running as tasks in the executors, and completed jobs transferring results back to the driver or writing to disk
 - The Spark application UI centralizes critical information, including status information. You can quickly identify failures, then drill down to the lowest levels of the application to discover their root causes
-